Diffusion Models I

Lecture 7: Denoising diffusion probabilistic models (DDPM)

Xingche Guo

STAT 5095 University of Connecticut · Fall 2026

VAE generated images

Sample $z \sim p(z)$, then use the decoder to generate an image.



- The model generates recognizable faces, but fine details can be blurred.
- A simple pixelwise decoder can favor smooth averages when details are uncertain.

Larsen et al. (2016), Fig. 3, VAE row. Original 64×64 CelebA samples. The next slide uses different data and resolutions.

DDPM generated images

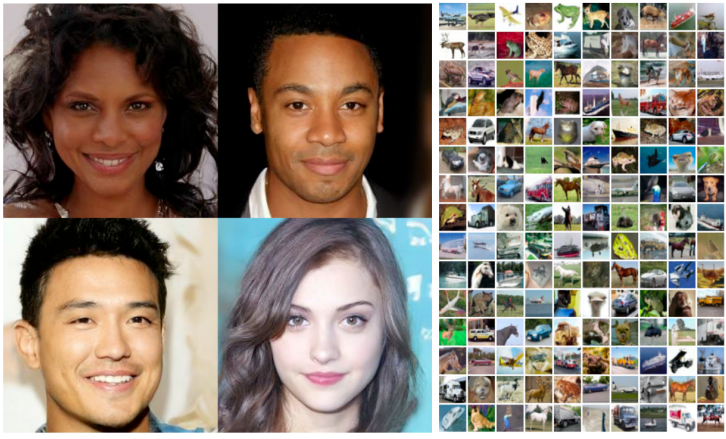


Figure 1: Generated samples on CelebA-HQ 256×256 (left) and unconditional CIFAR10 (right)

Ho, Jain & Abbeel (2020), Fig. 1. Generated CelebA-HQ faces and unconditional CIFAR-10 samples.

The diffusion idea

Forward: structure gradually disappears



Reverse: learn to build structure back

The key idea: Gradually add noise to real images, and learn to **reverse the process**, one small step at a time.

Generating a new image: Start from fresh Gaussian noise and apply the learned reverse steps to produce a **new image**.

The DDPM idea

Consider the forward diffusion process $x_0 \longrightarrow x_1 \longrightarrow \cdots \longrightarrow x_T$, where x_0 is the observed image and each step adds noise. Let $q_0(x_0)$ denote the data density and $q_t(x_t)$ the marginal density at time t. Under the **Markov assumption**, the joint density factorizes as

$$q(x_{0:T}) = q_0(x_0) \prod_{t=1}^{T} q(x_t \mid x_{t-1}).$$

By Bayes' rule,

$$q(x_{t-1} \mid x_t, \ldots, x_T) = \frac{q_{t-1}(x_{t-1})\, q(x_t \mid x_{t-1}) \prod_{s=t+1}^{T} q(x_s \mid x_{s-1})}{q_t(x_t) \prod_{s=t+1}^{T} q(x_s \mid x_{s-1})}$$

$$= \frac{q_{t-1}(x_{t-1})\, q(x_t \mid x_{t-1})}{q_t(x_t)} = q(x_{t-1} \mid x_t).$$

Thus, **the exact reverse process is also Markovian.** The same joint distribution admits the reverse factorization:

$$\boxed{q(x_{0:T}) = q_T(x_T) \prod_{t=1}^{T} q(x_{t-1} \mid x_t).}$$

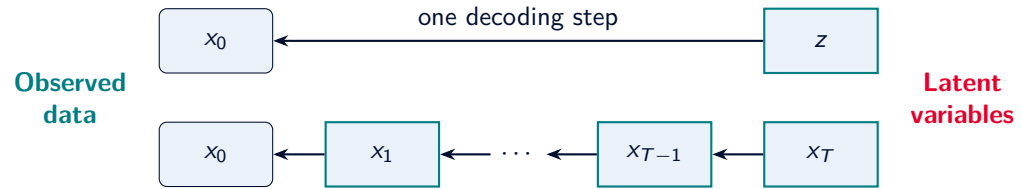
DDPM as a hierarchical VAE

VAE: one latent variable and one decoding step:

$$p_\theta(x_0, z) = p(z)\, p_\theta(x_0 \mid z).$$

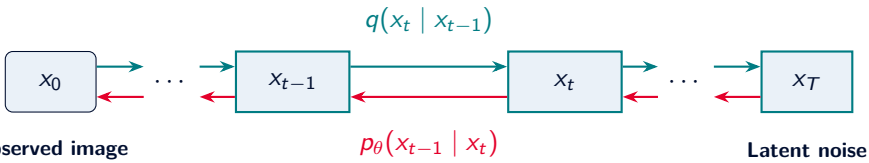
DDPM: a chain of latent states with learned reverse transitions:

$$p_\theta(x_{0:T}) = \underbrace{p(x_T)}_{\approx q_T(x_T)} \prod_{t=1}^{T} \underbrace{p_\theta(x_{t-1} \mid x_t)}_{\approx q(x_{t-1}|x_t)}.$$



Generation: Draw $x_T \sim \mathcal{N}(0, I_D)$, then generate an image x_0 through a sequence of learned denoising steps.

DDPM: learning to denoise



Forward process (fixed): Gradually add Gaussian noise using a schedule $\{\beta_t\}$.

$$q(x_t \mid x_{t-1}) = \mathcal{N}\left(\sqrt{1-\beta_t}\, x_{t-1},\, \beta_t I_D\right).$$

Reverse process (learning): Given a noisy state x_t, learn a distribution over plausible, less noisy states x_{t-1}:

$$\boxed{p_\theta(x_{t-1} \mid x_t) \approx q(x_{t-1} \mid x_t).}$$

Every state has the same dimension: $x_0, x_1, \ldots, x_T \in \mathbb{R}^D$.

The forward process

Choose a fixed noise schedule $0 < \beta_t < 1$ for $t = 1, \ldots, T$, and let

$$\alpha_t = 1 - \beta_t.$$

Each step shrinks the current state and adds Gaussian noise:

$$\boxed{q(x_t \mid x_{t-1}) = \mathcal{N}(x_t; \sqrt{\alpha_t}\, x_{t-1}, \beta_t I_D)\,.}$$

Equivalently, generate the next state by

$$x_t = \underbrace{\sqrt{\alpha_t}\, x_{t-1}}_{\text{retain part of the signal}} + \underbrace{\sqrt{\beta_t}\, \epsilon_t}_{\text{add noise}}, \qquad \epsilon_t \overset{\text{iid}}{\sim} \mathcal{N}(0, I_D),$$

where ϵ_t is independent of the previous states.

The scaling preserves the standard Gaussian distribution:

$$x_{t-1} \sim \mathcal{N}(0, I_D) \quad \Longrightarrow \quad x_t \sim \mathcal{N}(0, (\alpha_t + \beta_t) I_D) = \mathcal{N}(0, I_D).$$

The forward marginal

Condition on a fixed image x_0, each x_t is a deterministic term plus a linear combination of independent Gaussian noises, so $q(x_t \mid x_0)$ is Gaussian.

Let m_t and $v_t I_D$ denote its conditional mean and covariance. The forward transition gives

$$\begin{aligned} m_t &= \sqrt{\alpha_t}\, m_{t-1}, & m_0 &= x_0, \\ v_t &= \alpha_t v_{t-1} + \beta_t, & v_0 &= 0. \end{aligned}$$

Define $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$, with $\bar{\alpha}_0 = 1$. Since $\beta_t = 1 - \alpha_t$, the recursions yield

$$m_t = \sqrt{\bar{\alpha}_t}\, x_0, \qquad 1 - v_t = \alpha_t(1 - v_{t-1}) = \bar{\alpha}_t.$$

Therefore,

$$\boxed{q(x_t \mid x_0) = \mathcal{N}\left(x_t; \sqrt{\bar{\alpha}_t}\, x_0,\, (1 - \bar{\alpha}_t) I_D\right).}$$

Direct forward sampling

At any time step t, sample x_t directly from the clean image x_0:

$$\boxed{x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I_D).}$$

No intermediate states are needed.

Approaching standard Gaussian noise. The conditional distribution is

$$q(x_t \mid x_0) = \mathcal{N}\Big(\sqrt{\bar{\alpha}_t}\, x_0,\, (1-\bar{\alpha}_t) I_D\Big)\,.$$

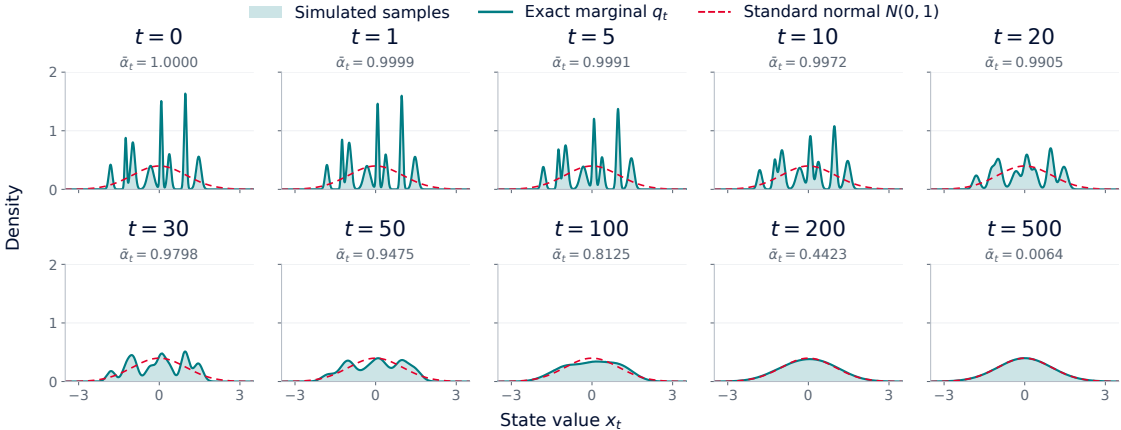
As $\bar{\alpha}_t \to 0$, its mean vanishes and its covariance approaches I_D, so

$$q(x_t \mid x_0) \longrightarrow \mathcal{N}(0, I_D).$$

For a finite number of steps, choose the noise schedule so that $\bar{\alpha}_T = \prod_{s=1}^{T}(1-\beta_s)$ is sufficiently small, giving $q(x_T \mid x_0) \approx \mathcal{N}(0, I_D)$.

An one-dimensional example

The forward process smooths an irregular, eight-component Gaussian mixture.



Python simulation: 150,000 trajectories, $T = 500$, and β_t increases linearly from 10^{-4} to 0.02. Initially, $\mathbb{E}[x_0] = 0$ and $\mathrm{Var}(x_0) = 1$. All panels use the same axes.

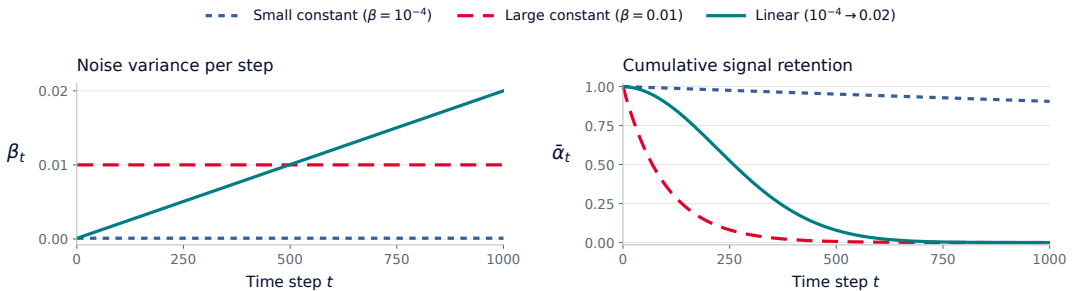
Choices of noise schedules

Constant: $\beta = 10^{-4}$ or 0.01.

$$\beta_t = \beta, \qquad \bar{\alpha}_t = (1-\beta)^t.$$

Linear: β_t increases with t.

$$\beta_t = \beta_{\min} + \frac{t-1}{T-1}(\beta_{\max} - \beta_{\min}).$$



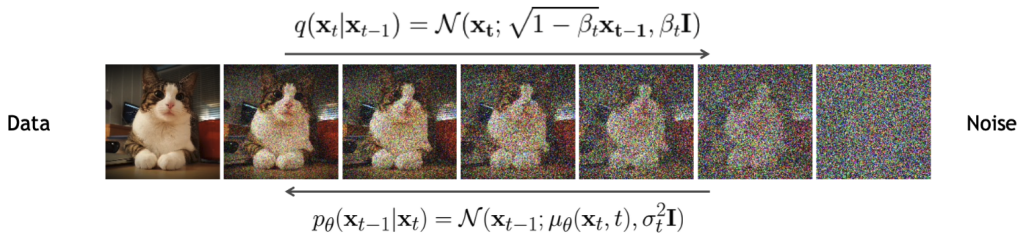
Small constant: small steps, but $\bar{\alpha}_{1000} \approx 0.905$; substantial signal remains.

Large constant: nearly pure noise by $T = 1000$, but the first noise SD is 10 times larger.

Linear: small initial steps and enough total noise: $\bar{\alpha}_{1000} \approx 4 \times 10^{-5}$.

Learning the reverse process

The forward transitions are known. Our goal is to **learn the reverse transitions**.



Define a **generative model** that starts from Gaussian noise:

$$p_\theta(x_{0:T}) = p(x_T) \prod_{t=1}^{T} p_\theta(x_{t-1} \mid x_t), \qquad p(x_T) = \mathcal{N}(0, I_D).$$

Here, x_0 is observed, while $x_{1:T}$ are **latent variables**.

How can we learn θ from observed images?

The Gaussian reverse process

Starting from Gaussian noise, define the **generative model**:

$$p_\theta(x_{0:T}) = p(x_T) \prod_{t=1}^{T} p_\theta(x_{t-1} \mid x_t), \qquad p(x_T) = \mathcal{N}(0, I_D).$$

For $t = T, \ldots, 2$, model each **reverse transition** as Gaussian:

$$p_\theta(x_{t-1} \mid x_t) = \mathcal{N}\Big(x_{t-1}; \mu_\theta(x_t, t), \sigma_t^2 I_D\Big).$$

- **Learned mean:** A shared neural network uses the noisy image x_t and time t to determine $\mu_\theta(x_t, t)$.
- **Fixed variance:** σ_t^2 follows a prespecified schedule and is not learned.

We will specify the mean parameterization and variance schedule later.

Variational inference for DDPM

Use the **fixed forward process** as the variational distribution for the latent path $x_{1:T}$:

$$q(x_{1:T} \mid x_0) = \prod_{t=1}^{T} q(x_t \mid x_{t-1}).$$

For an observed image x_0, lower-bound its log-likelihood:

$$\begin{aligned}
\log p_\theta(x_0) &= \log \int p_\theta(x_{0:T})\, dx_{1:T} \\
&= \log \mathbb{E}_{q(x_{1:T}|x_0)} \left[\frac{p_\theta(x_{0:T})}{q(x_{1:T} \mid x_0)} \right] \\
&\geq \underbrace{\mathbb{E}_{q(x_{1:T}|x_0)} \left[\log \frac{p_\theta(x_{0:T})}{q(x_{1:T} \mid x_0)} \right]}_{\text{ELBO}}.
\end{aligned}$$

Learning: maximize the ELBO over θ. The forward distribution q has **no trainable parameters**.

Rewriting forward distribution

To compare with the generative model, rewrite q using **reverse conditionals**.

$$
\begin{aligned}
q(x_{1:T} \mid x_0) &= q(x_1 \mid x_0) \prod_{t=2}^{T} q(x_t \mid x_{t-1}, x_0) \\
&\overset{\text{Bayes}}{=} q(x_1 \mid x_0) \prod_{t=2}^{T} \frac{q(x_t \mid x_0)\, q(x_{t-1} \mid x_t, x_0)}{q(x_{t-1} \mid x_0)} \\
&= \underbrace{q(x_1 \mid x_0) \prod_{t=2}^{T} \frac{q(x_t \mid x_0)}{q(x_{t-1} \mid x_0)}}_{q(x_T \mid x_0) \text{ (telescoping)}} \prod_{t=2}^{T} q(x_{t-1} \mid x_t, x_0).
\end{aligned}
$$

Thus,

$$
\boxed{q(x_{1:T} \mid x_0) = q(x_T \mid x_0) \prod_{t=2}^{T} q(x_{t-1} \mid x_t, x_0).}
$$

These reverse conditionals also use the **clean image** x_0, which is available during training.

The ELBO decomposition

Using the reverse factorization of q,

$$\frac{p_\theta(x_{0:T})}{q(x_{1:T} \mid x_0)} = p_\theta(x_0 \mid x_1)\,\frac{p(x_T)}{q(x_T \mid x_0)} \prod_{t=2}^{T} \frac{p_\theta(x_{t-1} \mid x_t)}{q(x_{t-1} \mid x_t, x_0)}.$$

Taking logs and expectations under $q(x_{1:T} \mid x_0)$ gives

$$\begin{aligned}
\mathcal{L}_{\mathrm{ELBO}}(\theta; x_0) &= \mathbb{E}_{q(x_1 \mid x_0)}[\log p_\theta(x_0 \mid x_1)] + \mathbb{E}_{q(x_T \mid x_0)}\left[\log \frac{p(x_T)}{q(x_T \mid x_0)}\right] \\
&\quad + \sum_{t=2}^{T} \mathbb{E}_{q(x_t, x_{t-1} \mid x_0)}\left[\log \frac{p_\theta(x_{t-1} \mid x_t)}{q(x_{t-1} \mid x_t, x_0)}\right] \\
&= \mathbb{E}_{q(x_1 \mid x_0)}[\log p_\theta(x_0 \mid x_1)] - \mathrm{KL}\left(q(x_T \mid x_0) \| p(x_T)\right) \\
&\quad + \sum_{t=2}^{T} \mathbb{E}_{q(x_t \mid x_0)} \underbrace{\mathbb{E}_{q(x_{t-1} \mid x_t, x_0)}\left[\log \frac{p_\theta(x_{t-1} \mid x_t)}{q(x_{t-1} \mid x_t, x_0)}\right]}_{-\mathrm{KL}(q(x_{t-1} \mid x_t, x_0) \| p_\theta(x_{t-1} \mid x_t))}.
\end{aligned}$$

The three parts of the ELBO

Maximizing the ELBO is equivalent to **minimizing its negative**:

$$
\begin{aligned}
-\mathcal{L}_{\mathrm{ELBO}}(\theta; x_0) = & \underbrace{-\mathbb{E}_{q(x_1|x_0)}\left[\log p_\theta(x_0 \mid x_1)\right]}_{L_0(x_0)\text{: Reconstruction}} \\
& + \underbrace{\mathrm{KL}\left(q(x_T \mid x_0) \,\|\, p(x_T)\right)}_{L_T(x_0)\text{: Prior matching}} \\
& + \sum_{t=2}^{T} \underbrace{\mathbb{E}_{q(x_t|x_0)}\left[\mathrm{KL}\left(q(x_{t-1} \mid x_t, x_0) \,\|\, p_\theta(x_{t-1} \mid x_t)\right)\right]}_{L_{t-1}(x_0)\text{: Denoising matching}}.
\end{aligned}
$$

Learning: Improve reconstruction and reduce the denoising KL divergences. The prior-matching term is **constant in** θ.

Three reverse distributions

For $t \geq 2$, distinguish the **true reverse transition**, the **training target**, and the **learned model**.

Distribution	Role	Key property
$q(x_{t-1} \mid x_t)$	True reverse transition	Usually intractable; not generally Gaussian
$q(x_{t-1} \mid x_t, x_0)$	Training target given the clean image x_0	Exactly Gaussian, with known mean and variance
$p_\theta(x_{t-1} \mid x_t)$	Learned approximation to the true reverse transition	Modeled as Gaussian; used for generation

The network receives only x_t and t in both training and generation. During training, x_0 supplies the denoising target.

Deriving the training target

For $t \geq 2$, Bayes' rule gives

$$q(x_{t-1} \mid x_t, x_0) = \frac{q(x_t \mid x_{t-1})\, q(x_{t-1} \mid x_0)}{q(x_t \mid x_0)}.$$

Both factors in the numerator are Gaussian:

$$q(x_t \mid x_{t-1}) = \mathcal{N}(x_t; \sqrt{\alpha_t}\, x_{t-1}, (1-\alpha_t) I_D)\,,$$

$$q(x_{t-1} \mid x_0) = \mathcal{N}\left(x_{t-1}; \sqrt{\bar{\alpha}_{t-1}}\, x_0, (1-\bar{\alpha}_{t-1}) I_D\right).$$

Therefore,

$$\log q(x_{t-1} \mid x_t, x_0) = -\frac{1}{2(1-\alpha_t)} \left\| x_t - \sqrt{\alpha_t}\, x_{t-1} \right\|^2 - \frac{1}{2(1-\bar{\alpha}_{t-1})} \left\| x_{t-1} - \sqrt{\bar{\alpha}_{t-1}}\, x_0 \right\|^2 + \text{Const.}$$

Deriving the training target

Define

$$a_t = \frac{1-\bar{\alpha}_t}{(1-\alpha_t)(1-\bar{\alpha}_{t-1})}, \qquad b_t = \frac{\sqrt{\alpha_t}}{1-\alpha_t}\, x_t + \frac{\sqrt{\bar{\alpha}_{t-1}}}{1-\bar{\alpha}_{t-1}}\, x_0.$$

Expanding and **completing the square**,

$$\log q(x_{t-1} \mid x_t, x_0) = -\frac{a_t}{2}\left\| x_{t-1} - \frac{b_t}{a_t} \right\|^2 + \text{Const.}$$

This gives us a Gaussian distribution:

$$\boxed{q(x_{t-1} \mid x_t, x_0) = \mathcal{N}\Big(x_{t-1}; \widetilde{\mu}_t(x_t, x_0), \widetilde{\beta}_t I_D\Big), \quad t \geq 2.}$$

where

$$\tilde{\beta}_t = \frac{1}{a_t} = \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\,\beta_t,$$

$$\tilde{\mu}_t(x_t, x_0) = \frac{b_t}{a_t} = \frac{\sqrt{\bar{\alpha}_{t-1}}\,\beta_t}{1-\bar{\alpha}_t}\, x_0 + \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\, x_t.$$

From Gaussian KL to squared-error loss

For $t \geq 2$, the target and learned reverse transition are Gaussian:

$$q(x_{t-1} \mid x_t, x_0) = \mathcal{N}\Big(x_{t-1}; \widetilde{\mu}_t(x_t, x_0), \widetilde{\beta}_t I_D\Big),$$

$$p_\theta(x_{t-1} \mid x_t) = \mathcal{N}\Big(x_{t-1}; \mu_\theta(x_t, t), \sigma_t^2 I_D\Big).$$

Suppressing the mean arguments, the Gaussian KL formula gives

$$\mathrm{KL}(q \,\|\, p_\theta) = \frac{D}{2}\left[\frac{\widetilde{\beta}_t}{\sigma_t^2} - 1 + \log\frac{\sigma_t^2}{\widetilde{\beta}_t}\right] + \frac{\|\widetilde{\mu}_t - \mu_\theta\|^2}{2\sigma_t^2}.$$

If we choose $\sigma_t^2 = \widetilde{\beta}_t$, the covariance terms cancel:

$$\boxed{\mathrm{KL}(q \,\|\, p_\theta) = \frac{1}{2\sigma_t^2}\,\|\widetilde{\mu}_t - \mu_\theta\|^2\,.}$$

Interpretation: Matching the reverse transitions reduces to a **weighted least squares problem** for the predicted mean.

From Gaussian KL to squared-error loss

The mean of $q(x_{t-1} \mid x_t, x_0)$ is a known linear combination of x_0 and x_t:

$$\widetilde{\mu}_t(x_t, x_0) = \frac{\sqrt{\bar{\alpha}_{t-1}}\,\beta_t}{1-\bar{\alpha}_t}\, x_0 + \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\, x_t.$$

Use the **same form** for the learned mean, replacing x_0 with a **network prediction** $\widetilde{x}_\theta(x_t, t)$:

$$\mu_\theta(x_t, t) = \frac{\sqrt{\bar{\alpha}_{t-1}}\,\beta_t}{1-\bar{\alpha}_t}\, \widetilde{x}_\theta(x_t, t) + \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\, x_t.$$

For $t \geq 2$ with $\sigma_t^2 = \widetilde{\beta}_t$, the common x_t term cancels when subtracting the means:

$$\begin{aligned} \mathrm{KL}(q \,\|\, p_\theta) &= \frac{1}{2\sigma_t^2} \left\| \widetilde{\mu}_t - \mu_\theta \right\|^2 \\ &= \frac{\bar{\alpha}_{t-1}\beta_t^2}{2\sigma_t^2(1-\bar{\alpha}_t)^2} \left\| x_0 - \widetilde{x}_\theta(x_t, t) \right\|^2 . \end{aligned}$$

From Gaussian KL to squared-error loss

For a **continuous Gaussian decoder** with $\sigma_1^2 > 0$, the reconstruction loss is

$$-\log p_\theta(x_0 \mid x_1) = \frac{1}{2\sigma_1^2} \|x_0 - \mu_\theta(x_1, 1)\|^2 + \text{Const.}$$

Using the empty-product convention $\bar{\alpha}_0 = 1$, our mean parameterization gives

$$\mu_\theta(x_1, 1) = \widetilde{x}_\theta(x_1, 1).$$

Combining reconstruction with the denoising losses,

$$-\mathcal{L}_{\text{ELBO}}(\theta; x_0) = \sum_{t=1}^{T} w_t \, \mathbb{E}_{q(x_t|x_0)} \left[\|x_0 - \widetilde{x}_\theta(x_t, t)\|^2 \right] + \text{Const},$$

with

$$w_t = \frac{\bar{\alpha}_{t-1}\beta_t^2}{2\sigma_t^2(1-\bar{\alpha}_t)^2}.$$

Maximizing the ELBO is equivalent to minimizing weighted squared error across noise levels.

From image prediction to noise prediction

We create a noisy training image by sampling $\epsilon \sim \mathcal{N}(0, I_D)$:

$$x_t = \underbrace{\sqrt{\bar{\alpha}_t}\, x_0}_{\text{signal}} + \underbrace{\sqrt{1-\bar{\alpha}_t}\, \epsilon}_{\text{noise}}.$$

If we knew the added noise, we could recover the clean image:

$$x_0 = \frac{x_t - \sqrt{1-\bar{\alpha}_t}\, \epsilon}{\sqrt{\bar{\alpha}_t}}.$$

DDPM learns to predict this noise. A neural network $\widetilde{\epsilon}_\theta(x_t, t)$ takes the noisy image and its noise level as inputs, giving

$$\widetilde{x}_\theta(x_t, t) = \frac{x_t - \sqrt{1-\bar{\alpha}_t}\, \widetilde{\epsilon}_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}}.$$

Thus, predicting the noise also provides an estimate of the clean image. At high noise levels, this estimate can be uncertain.

From image prediction to noise prediction

The two prediction errors differ only by a known scale:

$$\|x_0 - \widetilde{x}_\theta(x_t, t)\|^2 = \frac{1-\bar{\alpha}_t}{\bar{\alpha}_t} \|\epsilon - \widetilde{\epsilon}_\theta(x_t, t)\|^2 .$$

Thus, the clean-image loss weight w_t becomes

$$\lambda_t = w_t \frac{1-\bar{\alpha}_t}{\bar{\alpha}_t} = \frac{\beta_t^2}{2\sigma_t^2 \alpha_t (1-\bar{\alpha}_t)}.$$

The negative ELBO becomes a weighted noise-prediction loss:

$$-\mathcal{L}_{\mathrm{ELBO}}(\theta; x_0) = \sum_{t=1}^{T} \lambda_t \, \mathbb{E}_{\epsilon \sim \mathcal{N}(0, I_D)} \left[\|\epsilon - \widetilde{\epsilon}_\theta(x_t, t)\|^2 \right] + \text{Const.},$$

where $x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \epsilon$.

Learning: Use one shared network to predict the sampled noise at every noise level. The input t tells the network how much corruption to expect.

Simplified training loss

The ELBO gives a weighted noise-prediction objective:

$$\mathcal{L}_{\text{weighted}}(\theta) = \frac{1}{T}\sum_{t=1}^{T} \lambda_t \, \mathbb{E}_{x_0,\epsilon}\left[\left\|\epsilon - \widetilde{\epsilon}_\theta(x_t, t)\right\|^2\right], \qquad x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \epsilon.$$

1. Give all time steps equal weight. Set $\lambda_t = 1$. Empirically, this reweighting improved sample quality in the original DDPM experiments.

2. Express the average as an expectation. Let $t \sim \text{Uniform}\{1, \ldots, T\}$:

$$\boxed{\mathcal{L}_{\text{simple}}(\theta) = \mathbb{E}_{x_0,t,\epsilon}\left[\left\|\epsilon - \widetilde{\epsilon}_\theta(x_t, t)\right\|^2\right].}$$

Here $x_0 \sim q(x_0)$ and $\epsilon \sim \mathcal{N}(0, I_D)$, independently of t.

Practical benefit: Each training image needs only one randomly sampled time step to obtain an unbiased estimate of the loss.

DDPM Training

Repeat until convergence:

1. **Sample a minibatch of clean images:**

$$x_0^{(1)}, \ldots, x_0^{(B)} \quad \text{from the training data.}$$

2. **Sample a timestep and noise independently for each image:**

$$t_i \sim \text{Uniform}\{1, \ldots, T\}, \qquad \epsilon^{(i)} \sim \mathcal{N}(0, I_D), \quad i = 1, \ldots, B.$$

3. **Construct noisy images:**

$$x_{t_i}^{(i)} = \sqrt{\bar{\alpha}_{t_i}}\, x_0^{(i)} + \sqrt{1 - \bar{\alpha}_{t_i}}\, \epsilon^{(i)}.$$

4. **Take gradient descent step on:**

$$\nabla_\theta \frac{1}{B} \sum_{i=1}^{B} \left\| \epsilon^{(i)} - \epsilon_\theta\left(x_{t_i}^{(i)}, t_i\right) \right\|^2.$$

Key idea: One timestep per image. No need to simulate the entire forward chain.

How to sample new data

Use the learned **reverse (denoising) process**: start from pure noise $x_T \sim \mathcal{N}(0, I_D)$ and move toward x_0. At each step, sample from the Gaussian reverse transition:

$$x_{t-1} = \mu_\theta(x_t, t) + \sigma_t z_t, \qquad z_t \sim \mathcal{N}(0, I_D).$$

How does the network determine the reverse mean? Recall the re-parameterization:

$$\mu_\theta(x_t, t) = \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1-\bar{\alpha}_t}\widetilde{x}_\theta(x_t, t) + \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}x_t.$$

The predicted clean image is obtained from the predicted noise:

$$\widetilde{x}_\theta(x_t, t) = \frac{x_t - \sqrt{1-\bar{\alpha}_t}\,\widetilde{\epsilon}_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}}.$$

Substituting gives the **reverse transition mean**:

$$\boxed{\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\widetilde{\epsilon}_\theta(x_t, t)\right).}$$

DDPM sampling

1. **Start from pure Gaussian noise:** $x_T \sim \mathcal{N}(0, I_D)$.
2. **Apply the learned reverse steps.** For $t = T, T-1, \ldots, 1$, use $\widetilde{\epsilon}_\theta(x_t, t)$ to predict the noise in x_t and compute

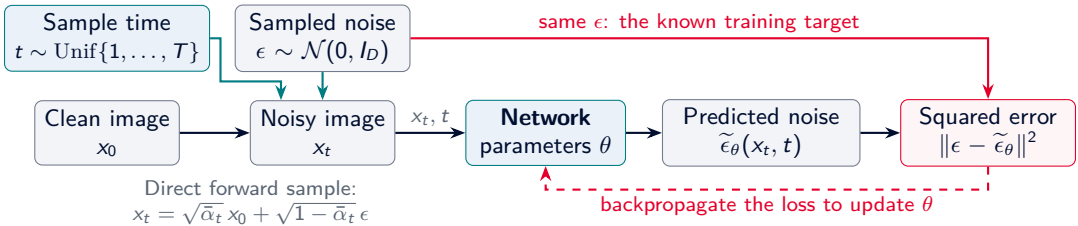
$$x_{t-1} = \underbrace{\frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\widetilde{\epsilon}_\theta(x_t, t)\right)}_{\text{learned reverse mean } \mu_\theta(x_t, t)} + \underbrace{\sigma_t z_t}_{\text{sampling noise}},$$

   where $z_t \sim \mathcal{N}(0, I_D)$ for $t > 1$ and $z_1 = 0$.
3. **Return x_0 as the generated image.**

Intuition: The learned mean guides each denoising step. The added noise reflects uncertainty about the previous state.

Training and generation

Training: create a noisy input and a known target



Generation: build a new image through the reverse chain

Keep the trained θ fixed



Every step reuses $\widetilde{\epsilon}_\theta(x_t, t)$. Add fresh Gaussian noise for $t > 1$, and none at $t = 1$.

Try DDPM on the CelebA data

The CelebA data

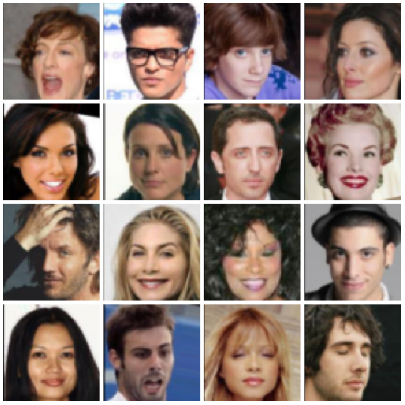
- CelebA contains **202,599 face images**.
- We uses **50,000 training**, 2,000 validation, and 2,000 test images.
- Crop to 148×148; resize to 64×64 RGB; normalize to $[-1, 1]$.

The task: unconditional generation

Learn the distribution of faces, then generate **new images from Gaussian noise**.

$$x_0 \in \mathbb{R}^{3\times 64\times 64}.$$

Real held-out CelebA examples

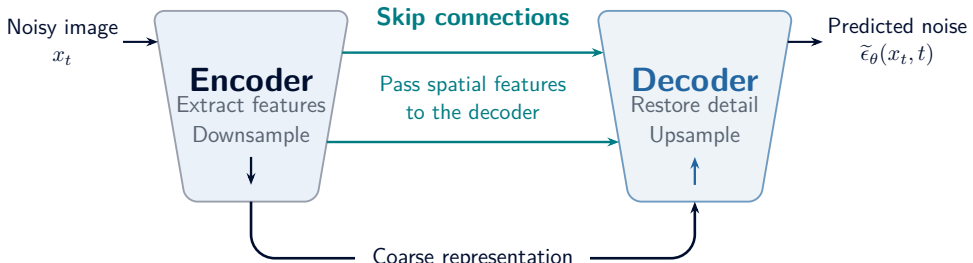


Data: Liu et al., CelebA (2015).

A U-Net for noise prediction

The original DDPM uses a **U-Net backbone** with **residual blocks**, a common choice for image diffusion that predicts noise with the **same dimensions** as the input image:

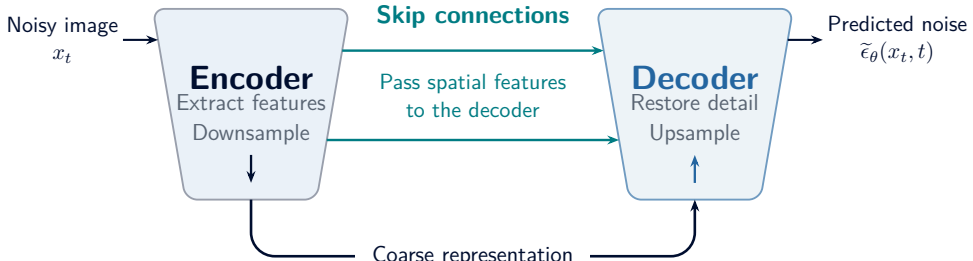
$$\underbrace{x_t \in \mathbb{R}^{3\times 64\times 64}}_{\text{noisy image}} \longmapsto \underbrace{\widetilde{\epsilon}_\theta(x_t, t) \in \mathbb{R}^{3\times 64\times 64}}_{\text{predicted noise: same shape}}.$$



A U-Net for noise prediction

The original DDPM uses a **U-Net backbone** with **residual blocks**, a common choice for image diffusion that predicts noise with the **same dimensions** as the input image:

$$\underbrace{x_t \in \mathbb{R}^{3\times 64\times 64}}_{\text{noisy image}} \longmapsto \underbrace{\widetilde{\epsilon}_\theta(x_t, t) \in \mathbb{R}^{3\times 64\times 64}}_{\text{predicted noise: same shape}}.$$



How do we incorporate the time step t into $\widetilde{\epsilon}_\theta(x_t, t)$?

Ho, Jain & Abbeel (2020).

Time as a sinusoidal embedding

We first **embed the time step t into a vector**, then **add learned transformations of this vector** to feature maps throughout the U-Net.

Embed the time step

Apply sinusoidal embedding on t:

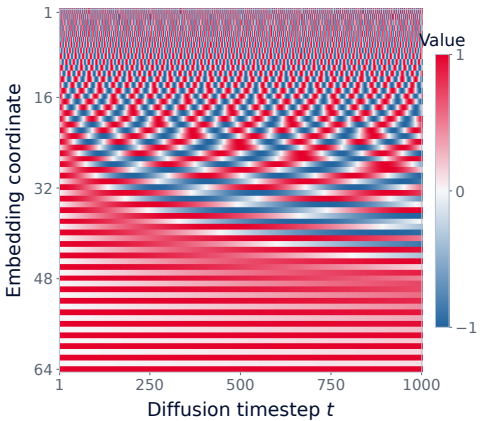
$$e(t) = \begin{bmatrix} \sin(\omega_1 t) \\ \cos(\omega_1 t) \\ \sin(\omega_2 t) \\ \cos(\omega_2 t) \\ \vdots \end{bmatrix} \in \mathbb{R}^{64},$$

where

$$\omega_k = 10000^{-(k-1)/31}$$

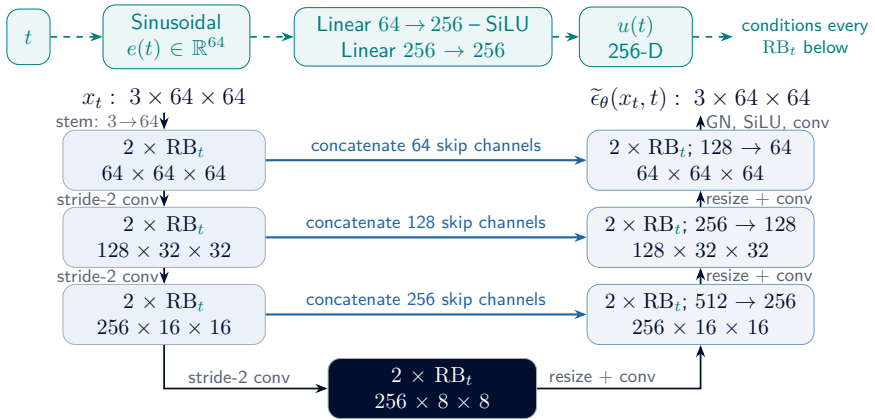
for $k = 1, \ldots, 32$.

$$E = [e(1), \ldots, e(1000)] \in \mathbb{R}^{64 \times 1000}.$$



Time-conditioned U-Net

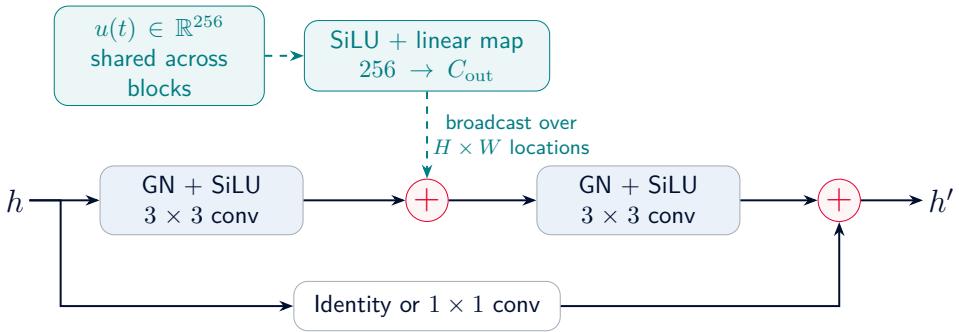
The implemented model: 14 residual blocks with 12.11M trainable parameters.



PyTorch documentation: GN (Group Normalization) SiLU (Sigmoid Linear Unit)

How time enters a residual block

Turn the time vector into a **channel-wise offset**, and add it to the feature map.

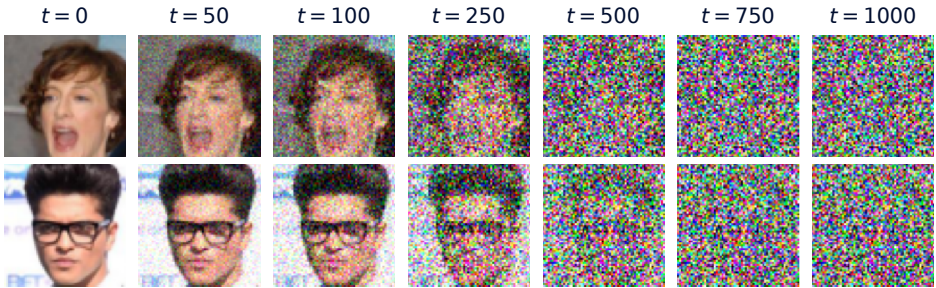


- The projected time embedding has C_{out} entries, matching the number of channels in h.
- Within each channel, the same offset is added at every spatial location (i, j).
- Each residual block learns its own projection of the time embedding.

Forward process: image to noise

We choose $T = 1000$. For any chosen timestep, sample directly from the forward marginal:

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1 - \bar{\alpha}_t}\, \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I).$$

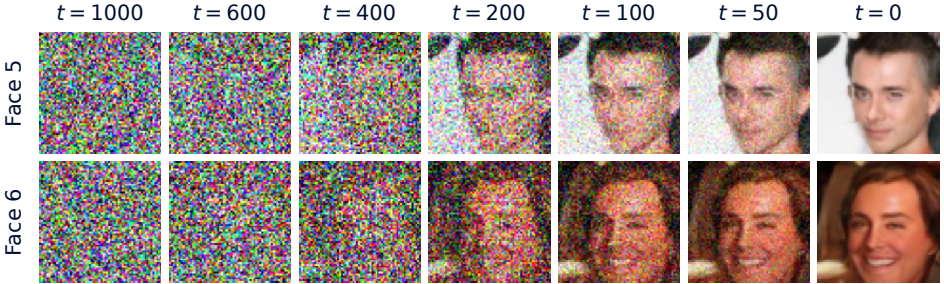


- A linear noise schedule is used: β_t increases linearly from 10^{-4} to 0.02.
- At $t = 1000$, $\sqrt{\bar{\alpha}_t} \approx 0.00635$: very little original signal remains.

Reverse process: noise to faces

Start with **fresh Gaussian noise**; use the trained U-Net to sample backward.

$$x_{1000} \sim \mathcal{N}(0, I) \quad \longrightarrow \quad x_{999} \quad \longrightarrow \quad \cdots \quad \longrightarrow \quad x_0.$$



- All 1,000 reverse steps are executed.
- Fresh noise is added for $t > 1$, and none at $t = 1$.

VAE vs DDPM generated samples

Both examples learn from 50,000 CelebA images at 64×64, for 30 epochs.

Convolutional VAE

$z \sim \mathcal{N}(0, I_{128}) \longrightarrow$ one decoder pass



These samples show recognizable faces with **smoother, blurrier textures**, a common tendency in standard VAEs.

DDPM with a U-Net

$x_{1000} \sim \mathcal{N}(0, I_{12288}) \longrightarrow$ 1,000 U-Net passes



These samples show **sharper hair and facial details**. Some artifacts remain, reflecting limited training data and model capacity.

Generalizing DDPM

DDPM Forward process: a Markov joint

$$q(x_{1:T} \mid x_0) = \prod_{t=1}^{T} q(x_t \mid x_{t-1}), \qquad q(x_t \mid x_{t-1}) = \mathcal{N}(\sqrt{\alpha_t} x_{t-1}, \beta_t I_D)\,.$$

Training: one marginal at a sampled time

$$q(x_t \mid x_0) = \mathcal{N}\Big(\sqrt{\bar{\alpha}_t} x_0, (1-\bar{\alpha}_t) I_D\Big)\,.$$

1. Sample independently

$$x_0 \sim q_{\text{data}}, \quad \epsilon \sim \mathcal{N}(0, I_D),$$
$$t \sim \text{Uniform}\{1, \ldots, T\}.$$
$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon.$$

2. Predict the added noise

$$\mathcal{L}_{\text{simple}}(\theta) = \mathbb{E}_{x_0,t,\epsilon}\Big[\big\|\epsilon - \widetilde{\epsilon}_\theta(x_t, t)\big\|^2\Big]\,.$$

Each evaluation uses (x_0, x_t). A full trajectory $x_{1:T}$ is unnecessary.

Do we really need the forward process to be Markov?

A joint with the same marginals

The DDPM joint admits a backward factorization:

$$q(x_{1:T} \mid x_0) = q(x_T \mid x_0) \prod_{t=2}^{T} q(x_{t-1} \mid x_t, x_0).$$

Generalize by choosing different conditional factors:

$$\boxed{q_\sigma(x_{1:T} \mid x_0) := q_\sigma(x_T \mid x_0) \prod_{t=2}^{T} q_\sigma(x_{t-1} \mid x_t, x_0).}$$

Preserve the same marginals. Keep the terminal distribution fixed:

$$q_\sigma(x_T \mid x_0) = q(x_T \mid x_0) = \mathcal{N}\left(\sqrt{\bar{\alpha}_T} x_0, (1 - \bar{\alpha}_T) I_D\right).$$

Choose the conditional factors so that

$$q_\sigma(x_t \mid x_0) = q(x_t \mid x_0), \qquad t = 1, \ldots, T-1.$$

The marginals can stay fixed while the dependence across time changes.

DDPM and generalized DDPM

Compare the forward processes:

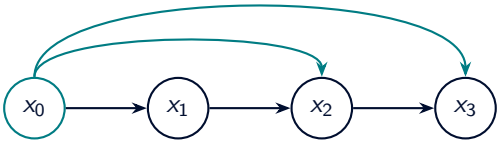
DDPM



$$q(x_t \mid x_{0:t-1}) = q(x_t \mid x_{t-1}).$$

The previous state is sufficient.
(Markovian)

Generalized DDPM



$$q_\sigma(x_t \mid x_{0:t-1}) = q_\sigma(x_t \mid x_{t-1}, x_0).$$

Can also depend on the clean image.
(non-Markovian)

Bayes' rule gives the forward conditional for $t \geq 2$:

$$q_\sigma(x_t \mid x_{t-1}, x_0) = \frac{q_\sigma(x_{t-1} \mid x_t, x_0)\, q_\sigma(x_t \mid x_0)}{q_\sigma(x_{t-1} \mid x_0)}.$$

Constructing the conditional

Lemma: For $a \in \mathbb{R}$, $x, y, b \in \mathbb{R}^D$, $\lambda, \ell > 0$:

$$x \sim \mathcal{N}(\mu, \lambda I_D), \quad y \mid x \sim \mathcal{N}(ax + b, \ell I_D) \quad \Longrightarrow \quad y \sim \mathcal{N}\Big(a\mu + b, (a^2\lambda + \ell) I_D\Big).$$

Apply the lemma conditional on x_0. For $t \geq 2$, choose

$$q_\sigma(x_{t-1} \mid x_t, x_0) = \mathcal{N}\Big(a_t x_t + b_t x_0,\ \tilde{\sigma}_t^2 I_D\Big).$$

Recall that:

$$q_\sigma(x_t \mid x_0) = \mathcal{N}\Big(\sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t) I_D\Big).$$

Matching the DDPM mean and covariance requires

$$a_t \sqrt{\bar{\alpha}_t} + b_t = \sqrt{\bar{\alpha}_{t-1}}, \qquad a_t^2 (1 - \bar{\alpha}_t) + \tilde{\sigma}_t^2 = 1 - \bar{\alpha}_{t-1}.$$

Taking $a_t \geq 0$ and $0 \leq \tilde{\sigma}_t^2 \leq 1 - \bar{\alpha}_{t-1}$ gives

$$\boxed{q_\sigma(x_{t-1} \mid x_t, x_0) = \mathcal{N}\left(\sqrt{\bar{\alpha}_{t-1}} x_0 + \sqrt{1 - \bar{\alpha}_{t-1} - \tilde{\sigma}_t^2}\, \frac{x_t - \sqrt{\bar{\alpha}_t} x_0}{\sqrt{1 - \bar{\alpha}_t}},\ \tilde{\sigma}_t^2 I_D\right).}$$

DDPM as a special case

For $t \geq 2$, choose the **DDPM posterior variance**:

$$\boxed{\tilde{\sigma}_t^2 = \widetilde{\beta}_t = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t}\,\beta_t.}$$

Substitution recovers the **DDPM posterior mean**:

$$\widetilde{\mu}_t(x_t, x_0) = \frac{\sqrt{\bar{\alpha}_{t-1}}\,\beta_t}{1 - \bar{\alpha}_t}\,x_0 + \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t}\,x_t.$$

The backward conditionals and the terminal distribution now agree with DDPM. Therefore, the **entire joint agrees**:

$$q_\sigma(x_{1:T} \mid x_0) = q(x_{1:T} \mid x_0).$$

Consequently, the forward transition is

$$q_\sigma(x_t \mid x_{t-1}, x_0) = q(x_t \mid x_{t-1}) = \mathcal{N}(\sqrt{\alpha_t}x_{t-1}, \beta_t I_D)\,.$$

This variance choice recovers the original Markov forward process.

DDIM: a deterministic special case

Denoising Diffusion Implicit Models (DDIM) use zero sampling variance:

$$\boxed{\tilde{\sigma}_t^2 = 0, \qquad t = 2, \dots, T.}$$

For $t \geq 2$, $q_\sigma(x_{t-1} \mid x_t, x_0)$ **degenerates to a point mass** at

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}}\, x_0 + \sqrt{1 - \bar{\alpha}_{t-1}}\, \frac{x_t - \sqrt{\bar{\alpha}_t}\, x_0}{\sqrt{1 - \bar{\alpha}_t}}.$$

Thus, given (x_t, x_0), the previous state x_{t-1} is determined.

The **terminal marginal remains unchanged**:

$$q_\sigma(x_T \mid x_0) = \mathcal{N}\Big(\sqrt{\bar{\alpha}_T}\, x_0,\, (1 - \bar{\alpha}_T) I_D\Big).$$

Given x_0, drawing x_T therefore determines the entire path $x_{T-1}, \dots, x_1$ through the **deterministic conditionals**.

Conditional on x_0, all randomness in the path comes from x_T.

Song, Meng & Ermon (2021), Denoising Diffusion Implicit Models, .

The shared training objective

The generalized forward process changes the dependence across time while preserving each **DDPM marginal**:

$$q_\sigma(x_t \mid x_0) = q(x_t \mid x_0) = \mathcal{N}\left(\sqrt{\bar{\alpha}_t}x_0, (1-\bar{\alpha}_t)I_D\right).$$

Training therefore uses the **same distribution of noisy images**. Therefore, the generalized DDPM uses the **same simplified noise prediction loss**:

$$\mathcal{L}_{\text{simple}}(\theta) = \mathbb{E}_{x_0,t,\epsilon}\left[\left\|\epsilon - \widetilde{\epsilon}_\theta\left(\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon, t\right)\right\|^2\right].$$

The derivation via variational inference gives **weighted** versions of this loss, with weights that may depend on σ. Using unit weights yields the same simplified objective.

A trained DDPM noise predictor can be reused for DDIM sampling.

For the derivation, see Song, Meng & Ermon (2021), DDIM, Sec. 3.2, Theorem 1.

A general sampling algorithm

The shared marginal expresses x_t as a combination of the clean image and noise:

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I_D).$$

Thus, **predicting the noise gives an estimate of** x_0.

Initialize $x_T \sim \mathcal{N}(0, I_D)$. For $t = T, \ldots, 1$, compute $\widehat{\epsilon}_t = \widetilde{\epsilon}_\theta(x_t, t)$ and update:

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \underbrace{\left(\frac{x_t - \sqrt{1-\bar{\alpha}_t}\, \widehat{\epsilon}_t}{\sqrt{\bar{\alpha}_t}} \right)}_{\text{predicted } x_0} + \underbrace{\sqrt{1-\bar{\alpha}_{t-1} - \tilde{\sigma}_t^2}\, \widehat{\epsilon}_t}_{\text{direction pointing to } x_t} + \underbrace{\tilde{\sigma}_t\, z_t}_{\text{fresh random noise}},$$

where $z_t \sim \mathcal{N}(0, I_D)$ is fresh noise drawn independently of x_t.

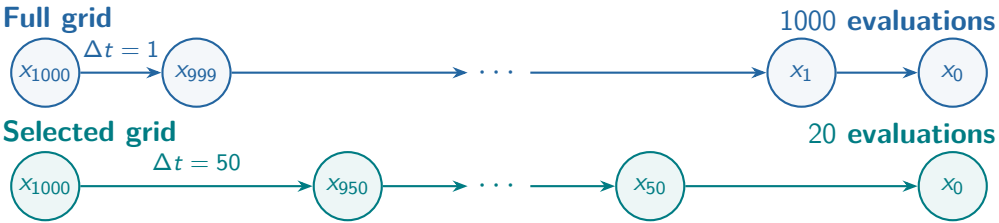
- **DDPM:** $\tilde{\sigma}_t^2 = \widetilde{\beta}_t$, giving stochastic updates for $t \geq 2$.
- **DDIM:** $\tilde{\sigma}_t = 0$, giving deterministic updates after the initial draw of x_T.

Set $\bar{\alpha}_0 = 1$ and $\tilde{\sigma}_1 = 0$ so the final step returns the predicted x_0.

Accelerated sampling

Motivation: Each reverse step requires a costly network evaluation.
Choose fewer times and **jump directly between them**:

$$\mathcal{T} = \{0 = t_0 < t_1 < \cdots < t_S = T\} \subseteq \{0, 1, \ldots, T\}, \qquad S \ll T.$$



Why can we skip steps? Training uses only the marginals $q(x_t \mid x_0)$. We preserve them at the selected times and construct conditionals that connect those times.

The shorter sampling path uses the same trained noise predictor.

Conditional between two times

Now consider the conditional distribution between any two times $1 \le t_1 < t_2 \le T$. First, note that $q_\sigma(x_{t_1} \mid x_{t_2}, x_0)$ is **Gaussian**, because it is obtained by composing **affine Gaussian kernels**:

$$q_\sigma(x_{t-2} \mid x_t, x_0) = \int q_\sigma(x_{t-2} \mid x_{t-1}, x_0) q_\sigma(x_{t-1} \mid x_t, x_0)\, \mathrm{d}x_{t-1}.$$

Apply the lemma again, for $1 \le t_1 < t_2 \le T$:

$$q_\sigma(x_{t_1} \mid x_{t_2}, x_0) = \mathcal{N}\Big(a_{12} x_{t_2} + b_{12} x_0, \tilde{\sigma}_{12}^2 I_D\Big), \qquad a_{12} \ge 0.$$

Matching the DDPM mean and covariance requires

$$a_{12}\sqrt{\bar{\alpha}_{t_2}} + b_{12} = \sqrt{\bar{\alpha}_{t_1}}, \qquad a_{12}^2(1 - \bar{\alpha}_{t_2}) + \tilde{\sigma}_{12}^2 = 1 - \bar{\alpha}_{t_1}.$$

This gives

$$\boxed{\begin{aligned} &q_\sigma(x_{t_1} \mid x_{t_2}, x_0) \\ &\quad = \mathcal{N}\left(\sqrt{\bar{\alpha}_{t_1}} x_0 + \sqrt{1 - \bar{\alpha}_{t_1} - \tilde{\sigma}_{12}^2}\, \frac{x_{t_2} - \sqrt{\bar{\alpha}_{t_2}} x_0}{\sqrt{1 - \bar{\alpha}_{t_2}}},\ \tilde{\sigma}_{12}^2 I_D\right). \end{aligned}}$$

Accelerated sampling algorithm

Choose $0 = t_0 < t_1 < \cdots < t_S = T$ and $\eta \in [0, 1]$. Initialize $x_{t_S} \sim \mathcal{N}(0, I_D)$.
For $i = S, \ldots, 1$, move directly from t_i to t_{i-1}:

1. Predict the noise and the clean image.

$$\widehat{\epsilon}_i = \widetilde{\epsilon}_\theta(x_{t_i}, t_i), \qquad \widehat{x}_0^{(i)} = \frac{x_{t_i} - \sqrt{1 - \bar{\alpha}_{t_i}}\,\widehat{\epsilon}_i}{\sqrt{\bar{\alpha}_{t_i}}}.$$

2. Set the jump variance, with $\bar{\alpha}_0 = 1$:

$$\tilde{\sigma}_i^2 = \eta^2 \, \frac{1 - \bar{\alpha}_{t_{i-1}}}{1 - \bar{\alpha}_{t_i}} \left(1 - \frac{\bar{\alpha}_{t_i}}{\bar{\alpha}_{t_{i-1}}}\right).$$

3. Update, using fresh independent noise $z_i \sim \mathcal{N}(0, I_D)$:

$$\boxed{x_{t_{i-1}} = \sqrt{\bar{\alpha}_{t_{i-1}}}\,\widehat{x}_0^{(i)} + \sqrt{1 - \bar{\alpha}_{t_{i-1}} - \tilde{\sigma}_i^2}\,\widehat{\epsilon}_i + \tilde{\sigma}_i z_i.}$$

$\eta = 0$: deterministic DDIM. $\eta = 1$: DDPM sampling on the selected grid.

Only S network evaluations, using the original time labels t_i.

DDIM: key advantages

Consistency across grids

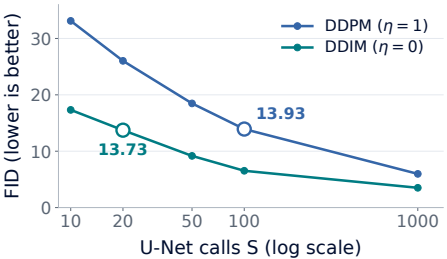
LSUN bedrooms



Same x_T in each column. Details change with S.

Quality with fewer steps

CelebA 64×64



Similar FID: DDIM 20 steps vs. DDPM 100.

- **Same training:** reuse the trained DDPM.
- **Deterministic:** fixed x_T and grid determine x_0.

Song, Meng & Ermon (2021), Denoising Diffusion Implicit Models, Left: crop of Fig. 5. Right: Table 1.

Appendix

Model training details

Group normalization (GN)

Normalize groups within each image.

One image: $C = 8$ channels, $G = 2$ groups



Each group pools its channels and $H \times W$.

$$y_{i,c,h,w} = \gamma_c \frac{h_{i,c,h,w} - \mu_{i,g}}{\sqrt{\sigma_{i,g}^2 + \varepsilon_{\text{num}}}} + \delta_c, \quad c \in g.$$

γ_c, δ_c: learned scale and offset.

Exponential moving average

Keep a smoothed copy of model weights. After optimizer update k:

$$\bar{\theta}_k = \rho_k \bar{\theta}_{k-1} + (1 - \rho_k)\theta_k.$$



Used in: **model training**

Used in: **model evaluation**

The original DDPM paper used EMA on model parameters with a decay factor of 0.9999.

Wu & He (2018), Group Normalization. Ho et al. (2020), DDPM, Appendix B.